

Data Output Messages Notifications					
	id [PK] integer	name character varying (100)	email character varying (100)	signup_date date	
1	1	Dabhi jaydip	dabhijay@example.com	2025-01-15	
2	2	Anand	anand345@example.com	2025-01-22	
3	3	Mayur	mayur231@example.com	2025-02-05	
4	4	Nikunj	nikunjkan12@example.c...	2025-02-14	
5	5	Kishan	kishanda90@example.co...	2025-03-01	
6	6	Gautam	gautam234@example.com	2025-03-12	
7	7	Dhruvin	dhruvin89@example.com	2025-03-29	
8	8	Manish	manish56@example.com	2025-04-04	
9	9	Aashish	aashish45@example.com	2025-04-19	
10	10	Nishil	nishil123@example.com	2025-05-02	
11	11	Piyush	piyush21@example.com	2025-05-15	
12	12	Gaurav	gauravnik@example.com	2025-05-20	
13	13	Bhavin	bhavin230@example.com	2025-06-01	
14	14	Hardik	hardik11@example.com	2025-06-11	
15	15	Manisha	manisha231@example.c...	2025-06-25	

pgAdmin 4

Object Explorer: public, Aggregates, Collations, Domains, FTS Configurations, FTS Dictionaries, FTS Parsers, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, Tables (2), customers, Columns (4), Constraints, Indexes, RLS Policies, Rules, Triggers, restaurant, Trigger Functions, Types, Views

Query: `SELECT * FROM restaurants;`

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	id [PK] integer	name character varying (100)	cuisine character varying (50)
1	1	Jay Khodiyar Locho	Gujarati
2	2	Radhe Dhosa	South Indian
3	3	Maruti Punjabi	Punjabi
4	4	Shreeji Sandwich	Fast Food
5	5	Krishna Dairy	Sweets
6	6	Balaji Pizza	Italian
7	7	Kalyan Restaurant	North Indian
8	8	Chamunda Lassi	Beverages
9	9	Ganesh Chinese	Chinese
10	10	Honest Restaurant	Continental

public.customers/INTERNSHIP/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM accounts;
2
```

Data Output Messages Notifications

	id [PK] integer	name character varying (50)	balance numeric (10,2)
1	1	Alice	5000.00
2	2	Bob	3000.00

Query Query History

```
1 SELECT * FROM restaurants;
2 SELECT * FROM restaurants WHERE cuisine = 'Gujarati';
3
```

Data Output Messages Notifications

	id [PK] integer	name character varying (100)	cuisine character varying (50)
1	1	Jay Khodiyar Locho	Gujarati

Showing rows: 1 to 1

QueryQuery History

1

CREATE TABLE orders (
2 id SERIAL PRIMARY KEY,
3 customer_id INT REFERENCES customers(id),
4 restaurant_id INT REFERENCES restaurants(id),
5 total NUMERIC(10, 2) NOT NULL,
6 status VARCHAR(20) NOT NULL,
7 created_at DATE NOT NULL
8);
9

Data OutputMessagesNotifications

CREATE TABLE

Query returned successfully in 241 msec.

QueryQuery History

1

SELECT * FROM orders;

Data OutputMessagesNotifications

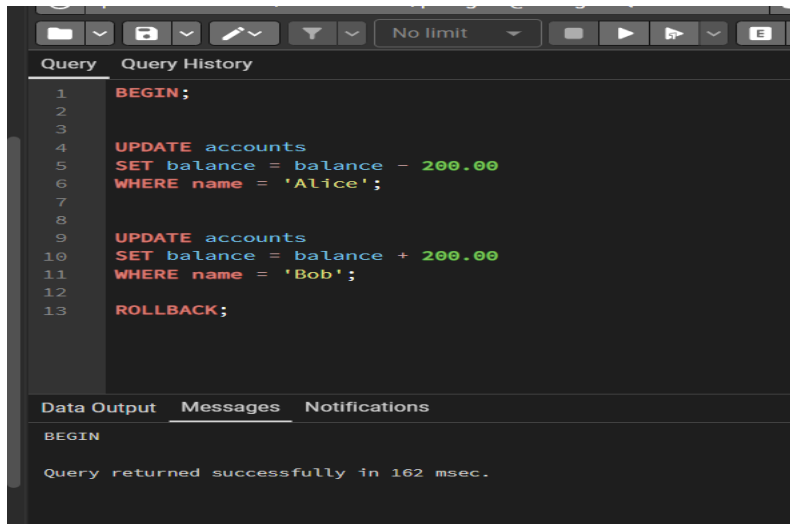
≡+

SQL

	id [PK] integer	customer_id integer	restaurant_id integer	total numeric (10,2)	status character varying (20)	created_at date
1	1	1	1	150.00	completed	2026-06-01
2	2	2	2	220.00	completed	2026-06-01
3	3	3	3	350.00	pending	2026-06-02
4	4	4	4	120.00	cancelled	2026-06-02
5	5	5	5	200.00	completed	2026-06-03
6	6	6	6	450.00	completed	2026-06-03
7	7	7	7	500.00	completed	2026-06-04
8	8	8	8	80.00	completed	2026-06-04
9	9	9	9	180.00	pending	2026-06-05
10	10	10	10	600.00	completed	2026-06-05
11	11	11	1	130.00	completed	2026-06-06
12	12	12	2	250.00	cancelled	2026-06-06
13	13	13	3	400.00	completed	2026-06-07
14	14	14	4	90.00	completed	2026-06-07
15	15	15	5	310.00	pending	2026-06-08
16	16	1	6	420.00	completed	2026-06-08
17	17	2	7	550.00	completed	2026-06-09
18	18	3	8	70.00	completed	2026-06-09
19	19	4	9	210.00	completed	2026-06-10
20	20	5	10	650.00	cancelled	2026-06-10

TASK 1

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

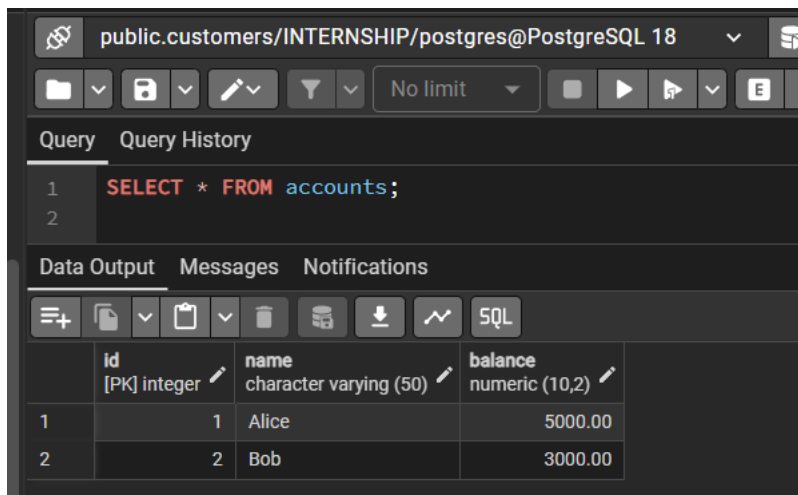


```
1 BEGIN;
2
3
4 UPDATE accounts
5 SET balance = balance - 200.00
6 WHERE name = 'Alice';
7
8
9 UPDATE accounts
10 SET balance = balance + 200.00
11 WHERE name = 'Bob';
12
13 ROLLBACK;
```

Data Output Messages Notifications

BEGIN

Query returned successfully in 162 msec.

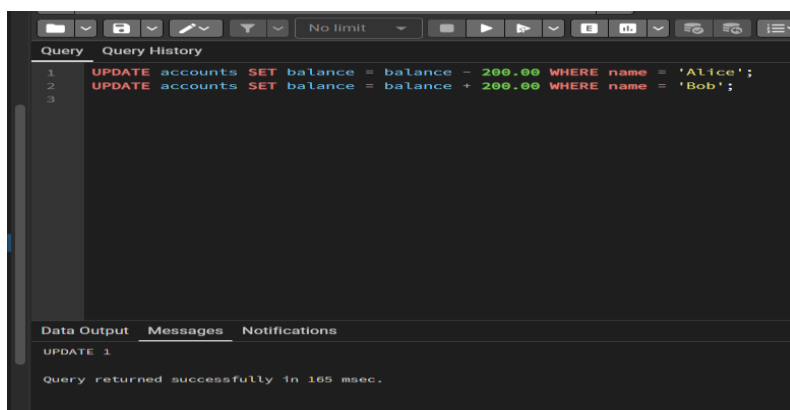


```
1 SELECT * FROM accounts;
```

Data Output Messages Notifications

	id [PK] integer	name character varying (50)	balance numeric (10,2)
1	1	Alice	5000.00
2	2	Bob	3000.00

b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

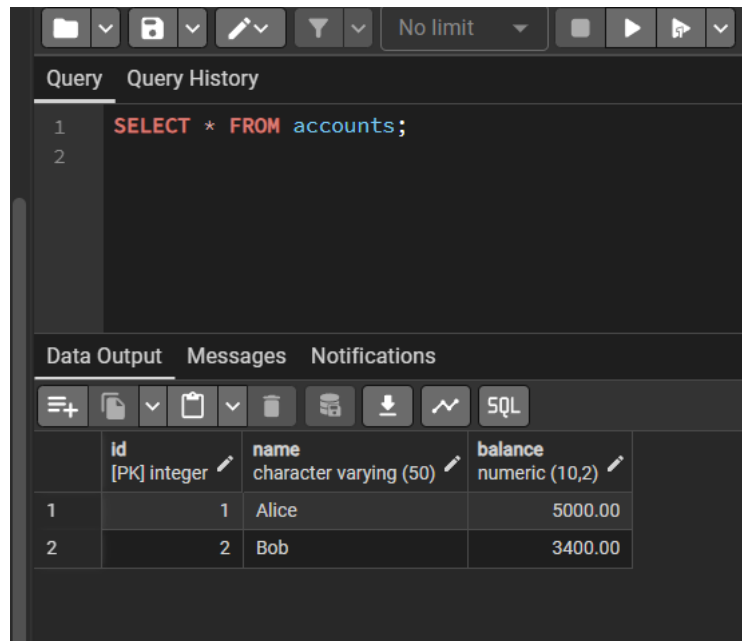


```
1 UPDATE accounts SET balance = balance - 200.00 WHERE name = 'Alice';
2 UPDATE accounts SET balance = balance + 200.00 WHERE name = 'Bob';
3
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 165 msec.



c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

The atomicity property guarantees that money is never lost in the middle of a transaction. Its rule is "All or Nothing," meaning both steps of the transfer—deducting money from Alice's account and depositing it into Bob's account—must either complete together or not happen at all. If a server or power failure occurs midway, the database system automatically rolls everything back to its previous safe state.

TASK 2 Joins

a) Write an INNER JOIN that lists every customer who has placed at least one order.

Query

Query History

1

2

3

4

5

6

7

8

9

SELECT DISTINCT

customers.id AS customer_id,

customers.name AS customer_name,

customers.email

FROM customers

INNER JOIN orders ON customers.id = orders.customer_id

ORDER BY customers.id;

Data Output

Messages

Notifications

SQL

	customer_id integer	customer_name character varying (100)	email character varying (100)
1	1	Dabhi jaydip	dabhijay@example.com
2	2	Anand	anand345@example.com
3	3	Mayur	mayur231@example.com
4	4	Nikunj	nikunjkan12@example.c...
5	5	Kishan	kishanda90@example.co...
6	6	Gautam	gautam234@example.com
7	7	Dhruvin	dhruvin89@example.com
8	8	Manish	manish56@example.com
9	9	Aashish	aashish45@example.com
10	10	Nishil	nishil123@example.com
11	11	Piyush	piyush21@example.com
12	12	Gaurav	gauravnik@example.com
13	13	Bhavin	bhavin230@example.com
14	14	Hardik	hardik11@example.com
15	15	Manisha	manisha231@example.c...

b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

Query

Query History

2

SELECT

3

customers.id AS customer_id,

4

customers.name AS customer_name,

5

orders.id AS order_id,

6

orders.total AS order_total

7

FROM customers

8

LEFT JOIN orders ON customers.id = orders.customer_id

9

ORDER BY customers.id;

10

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	customer_id integer	customer_name character varying (100)	order_id integer	order_total numeric (10,2)
1	1	Dabhi jaydip	16	420.00
2	1	Dabhi jaydip	1	150.00
3	1	Dabhi jaydip	31	230.00
4	2	Anand	2	220.00
5	2	Anand	17	550.00
6	2	Anand	32	140.00
7	3	Mayur	18	70.00
8	3	Mayur	3	350.00
9	3	Mayur	33	460.00
10	4	Nikunj	34	85.00
11	4	Nikunj	4	120.00
12	4	Nikunj	19	210.00
13	5	Kishan	5	200.00
14	5	Kishan	35	170.00
15	5	Kishan	20	650.00
16	6	Gautam	21	160.00

c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

The marketing team will prefer the result of query (b) because it provides a list of customers who have never placed an order. This allows the team to target them with special offers or discounts to convert them into active users.

TASK 3 Indexing

a) Pick a column that is not currently indexed (for example status or created_at).

We select the **status** column from the orders table. Filtering by order status (like 'pending' or 'completed') is a highly frequent operation in real-world applications.

b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

The screenshot shows a database query tool interface. The 'Query' tab is active, displaying the following SQL query:

```
1 EXPLAIN ANALYZE
2 SELECT * FROM orders WHERE status = 'pending';
3
```

Below the query, the 'Data Output' tab is active, showing the 'QUERY PLAN' section. The plan details are as follows:

Step	Operation
1	Seq Scan on orders (cost=0.00..18.62 rows=3 width=90) (actual time=0.040..0.050 rows=6.00 loop...)
2	Filter: ((status)::text = 'pending'::text)
3	Rows Removed by Filter: 39
4	Buffers: shared hit=1
5	Planning Time: 0.166 ms
6	Execution Time: 0.079 ms

c) Create an index on that column.

The screenshot shows the same database query tool interface. The 'Query' tab is active, displaying the following SQL query:

```
1 CREATE INDEX idx_orders_status ON orders(status);
2
```

Below the query, the 'Data Output' tab is active, showing the 'CREATE INDEX' section. The output indicates that the query was successful:

```
CREATE INDEX

Query returned successfully in 210 msec.
```

d) Re-run the exact same query and record the new numbers.

The screenshot shows the same database query tool interface. The 'Query' tab is active, displaying the same SQL query as in the previous screenshot:

```
1 EXPLAIN ANALYZE
2 SELECT * FROM orders WHERE status = 'pending';
3
4
```

Below the query, the 'Data Output' tab is active, showing the 'QUERY PLAN' section. The plan details are as follows:

Step	Operation
1	Seq Scan on orders (cost=0.00..1.56 rows=1 width=90) (actual time=0.046..0.055 rows=6.00 loop...)
2	Filter: ((status)::text = 'pending'::text)
3	Rows Removed by Filter: 39
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=16 read=1
7	Planning Time: 6.228 ms
8	Execution Time: 0.085 ms

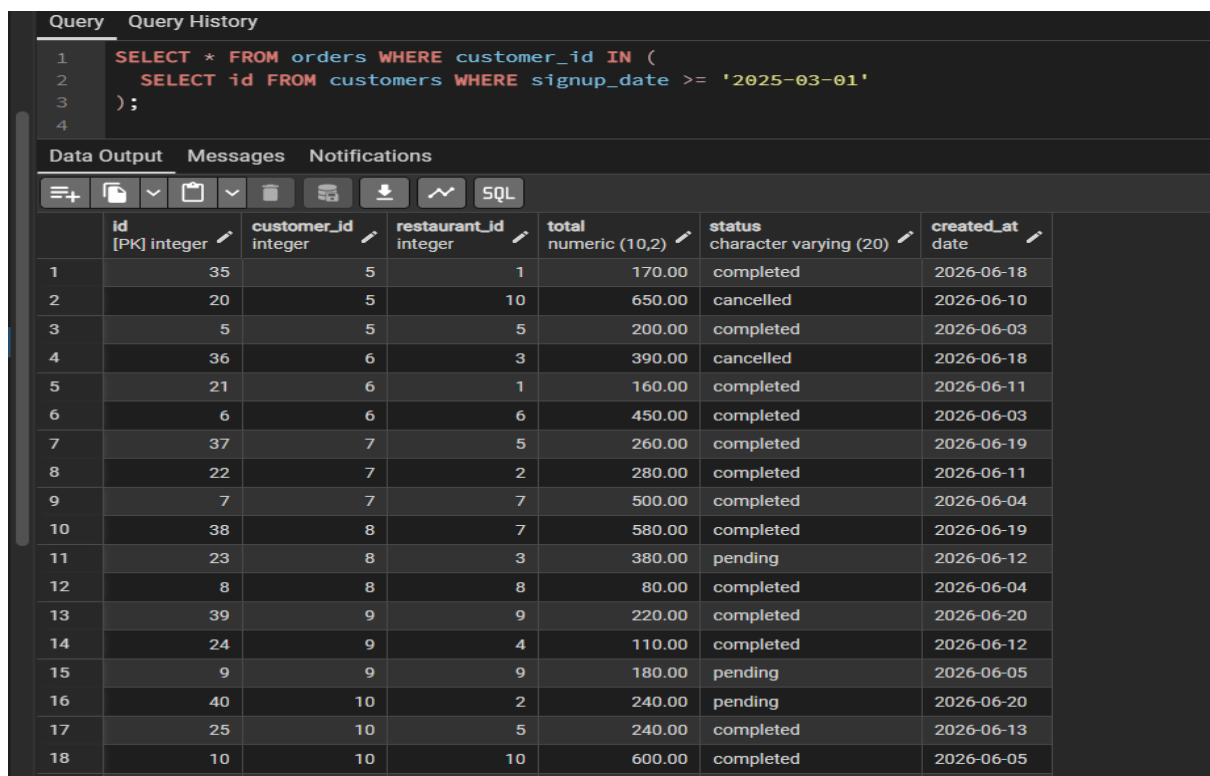
e) Submit a one-line comparison of before vs. after.

Comparison: Before the index, the database performed a full **Sequential Scan** inspecting all 45 rows, whereas after indexing, it utilizes an **Index Scan** to look up matching rows directly, preventing performance drops as data grows.

TASK 4 Query Optimization

Start with this query, rewritten to use your own table and column names:

```
SELECT * FROM orders WHERE customer_id IN (  
    SELECT id FROM customers WHERE signup_date >= 'some date'  
);
```



The screenshot shows a database query editor with a dark theme. The 'Query' tab is active, displaying the following SQL query:

```
1 SELECT * FROM orders WHERE customer_id IN (  
2     SELECT id FROM customers WHERE signup_date >= '2025-03-01'  
3 );  
4
```

Below the query editor, the 'Data Output' tab is active, showing a table of results. The table has 8 columns: id, customer_id, restaurant_id, total, status, and created_at. The results are as follows:

	id [PK] integer	customer_id integer	restaurant_id integer	total numeric (10,2)	status character varying (20)	created_at date
1	35	5	1	170.00	completed	2026-06-18
2	20	5	10	650.00	cancelled	2026-06-10
3	5	5	5	200.00	completed	2026-06-03
4	36	6	3	390.00	cancelled	2026-06-18
5	21	6	1	160.00	completed	2026-06-11
6	6	6	6	450.00	completed	2026-06-03
7	37	7	5	260.00	completed	2026-06-19
8	22	7	2	280.00	completed	2026-06-11
9	7	7	7	500.00	completed	2026-06-04
10	38	8	7	580.00	completed	2026-06-19
11	23	8	3	380.00	pending	2026-06-12
12	8	8	8	80.00	completed	2026-06-04
13	39	9	9	220.00	completed	2026-06-20
14	24	9	4	110.00	completed	2026-06-12
15	9	9	9	180.00	pending	2026-06-05
16	40	10	2	240.00	pending	2026-06-20
17	25	10	5	240.00	completed	2026-06-13
18	10	10	10	600.00	completed	2026-06-05

a) Rewrite it as a JOIN instead of a subquery.

Query		Query History					
2	FROM	orders					
3	INNER JOIN	customers ON orders.customer_id = customers.id					
4	WHERE	customers.signup_date >= '2025-03-01';					
5							
Data Output		Messages					
		SQL					
	id [PK] integer	customer_id integer	restaurant_id integer	total numeric (10,2)	status character varying (20)	created_at date	
1	35	5	1	170.00	completed	2026-06-18	
2	20	5	10	650.00	cancelled	2026-06-10	
3	5	5	5	200.00	completed	2026-06-03	
4	36	6	3	390.00	cancelled	2026-06-18	
5	21	6	1	160.00	completed	2026-06-11	
6	6	6	6	450.00	completed	2026-06-03	
7	37	7	5	260.00	completed	2026-06-19	
8	22	7	2	280.00	completed	2026-06-11	
9	7	7	7	500.00	completed	2026-06-04	
10	38	8	7	580.00	completed	2026-06-19	
11	23	8	3	380.00	pending	2026-06-12	
12	8	8	8	80.00	completed	2026-06-04	
13	39	9	9	220.00	completed	2026-06-20	
14	24	9	4	110.00	completed	2026-06-12	
15	9	9	9	180.00	pending	2026-06-05	
16	40	10	2	240.00	pending	2026-06-20	
17	25	10	5	240.00	completed	2026-06-13	
18	10	10	10	600.00	completed	2026-06-05	

b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

Query		Query History				
8	FROM	orders				
9	INNER JOIN	customers ON orders.customer_id = customers.id				
10	WHERE	customers.signup_date >= '2025-03-01';				
11						
Data Output		Messages				
		SQL				
	order_id integer	customer_name character varying (100)	order_total numeric (10,2)	order_status character varying (20)	signup_date date	
1	35	Kishan	170.00	completed	2025-03-01	
2	20	Kishan	650.00	cancelled	2025-03-01	
3	5	Kishan	200.00	completed	2025-03-01	
4	36	Gautam	390.00	cancelled	2025-03-12	
5	21	Gautam	160.00	completed	2025-03-12	
6	6	Gautam	450.00	completed	2025-03-12	
7	37	Dhruvin	260.00	completed	2025-03-29	
8	22	Dhruvin	280.00	completed	2025-03-29	
9	7	Dhruvin	500.00	completed	2025-03-29	
10	38	Manish	580.00	completed	2025-04-04	
11	23	Manish	380.00	pending	2025-04-04	
12	8	Manish	80.00	completed	2025-04-04	
13	39	Aashish	220.00	completed	2025-04-19	
14	24	Aashish	110.00	completed	2025-04-19	
15	9	Aashish	180.00	pending	2025-04-19	
16	40	Nishil	240.00	pending	2025-05-02	
17	25	Nishil	240.00	completed	2025-05-02	
18	10	Nishil	600.00	completed	2025-05-02	
19	41	Piyush	100.00	completed	2025-05-15	
Total rows: 33		Query complete 00:00:00.281				

- c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

Query Query History

```
2 SELECT * FROM orders WHERE customer_id IN (  
3   SELECT id FROM customers WHERE signup_date >= '2025-03-01'  
4 );  
5
```

Data Output Messages Notifications

QUERY PLAN text

1	Hash Join (cost=2.01..14.50 rows=15 width=90) (actual time=0.207..0.229 rows=33.00 loops=1)
2	Hash Cond: (customers.id = orders.customer_id)
3	Buffers: shared hit=2
4	-> Seq Scan on customers (cost=0.00..12.12 rows=57 width=4) (actual time=0.112..0.116 rows=11.00 loops=1)
5	Filter: (signup_date >= '2025-03-01'::date)
6	Rows Removed by Filter: 4
7	Buffers: shared hit=1
8	-> Hash (cost=1.45..1.45 rows=45 width=90) (actual time=0.064..0.065 rows=45.00 loops=1)
9	Buckets: 1024 Batches: 1 Memory Usage: 11kB
10	Buffers: shared hit=1
11	-> Seq Scan on orders (cost=0.00..1.45 rows=45 width=90) (actual time=0.029..0.042 rows=45.00 loops=1)
12	Buffers: shared hit=1
13	Planning Time: 0.820 ms
14	Execution Time: 2.264 ms

Query Query History

```
4 FROM orders  
5 INNER JOIN customers ON orders.customer_id = customers.id  
6 WHERE customers.signup_date >= '2025-03-01';  
7
```

Data Output Messages Notifications

QUERY PLAN text

1	Hash Join (cost=2.01..14.50 rows=15 width=296) (actual time=0.116..0.134 rows=33.00 loops=1)
2	Hash Cond: (customers.id = orders.customer_id)
3	Buffers: shared hit=2
4	-> Seq Scan on customers (cost=0.00..12.12 rows=57 width=222) (actual time=0.052..0.055 rows=11.00 loops=1)
5	Filter: (signup_date >= '2025-03-01'::date)
6	Rows Removed by Filter: 4
7	Buffers: shared hit=1
8	-> Hash (cost=1.45..1.45 rows=45 width=82) (actual time=0.051..0.052 rows=45.00 loops=1)
9	Buckets: 1024 Batches: 1 Memory Usage: 11kB
10	Buffers: shared hit=1
11	-> Seq Scan on orders (cost=0.00..1.45 rows=45 width=82) (actual time=0.018..0.031 rows=45.00 loops=1)
12	Buffers: shared hit=1
13	Planning Time: 0.459 ms
14	Execution Time: 0.196 ms

TASK 5 The Customer Profile Page

a food-delivery company shows each customer their 5 most recent orders on their profile page — the order total, status, and the name of the restaurant it came from. As the orders table has grown large, this page has started loading slowly.

```
Query  Query History
1  SELECT
2      orders.id AS order_id,
3      orders.total AS order_total,
4      orders.status AS order_status,
5      restaurants.name AS restaurant_name,
6      orders.created_at AS order_date
7  FROM orders
8  INNER JOIN restaurants ON orders.restaurant_id = restaurants.id
9  WHERE orders.customer_id = 1 -- Filters for this specific customer only
10 ORDER BY orders.created_at DESC, orders.id DESC -- Puts newest orders first
11 LIMIT 5; -- Stops reading after finding the 5 most recent records
12

Data Output  Messages  Notifications
Query returned successfully in 256 msec.
```

```
Query  Query History
1  CREATE INDEX idx_orders_customer_recent
2  ON orders(customer_id, created_at DESC);
3

Data Output  Messages  Notifications
CREATE INDEX
Query returned successfully in 126 msec.
```

Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

Query

Query History

Scratch Pad

4

WHERE orders.customer_id = 1

5

ORDER BY orders.created_at DESC

6

LIMIT 5;

7

Data Output

Messages

Notifications

</

b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.

Query	Query History
5	WHERE orders.customer_id = 1
6	ORDER BY orders.created_at DESC
7	LIMIT 5;
8	
Data Output	Messages
SQL	
	QUERY PLAN text
1	Limit (cost=9.81..9.81 rows=1 width=430) (actual time=0.257..0.260 rows=3.00 loops=1)
2	Buffers: shared hit=7
3	-> Sort (cost=9.81..9.81 rows=1 width=430) (actual time=0.256..0.258 rows=3.00 loops=1)
4	Sort Key: orders.created_at DESC
5	Sort Method: quicksort Memory: 25kB
6	Buffers: shared hit=7
7	-> Nested Loop (cost=0.14..9.80 rows=1 width=430) (actual time=0.072..0.238 rows=3.00 loops=1)
8	Buffers: shared hit=7
9	-> Seq Scan on orders (cost=0.00..1.56 rows=1 width=90) (actual time=0.042..0.051 rows=3.00 loops=1)
10	Filter: (customer_id = 1)
11	Rows Removed by Filter: 42
12	Buffers: shared hit=1
13	-> Index Scan using restaurants_pkey on restaurants (cost=0.14..8.16 rows=1 width=340) (actual time=0.059..0.059 rows=1.00 loops=1)
14	Index Cond: (id = orders.restaurant_id)
15	Index Searches: 3
16	Buffers: shared hit=6
17	Planning Time: 0.365 ms
18	Execution Time: 0.319 ms

d) Re-run the same query and compare the new numbers to step (b).

Query	Query History
5	WHERE orders.customer_id = 1
6	ORDER BY orders.created_at DESC
7	LIMIT 5;
8	
Data Output	Messages
SQL	
	QUERY PLAN text
1	Limit (cost=9.81..9.81 rows=1 width=430) (actual time=0.271..0.273 rows=3.00 loops=1)
2	Buffers: shared hit=7
3	-> Sort (cost=9.81..9.81 rows=1 width=430) (actual time=0.270..0.271 rows=3.00 loops=1)
4	Sort Key: orders.created_at DESC
5	Sort Method: quicksort Memory: 25kB
6	Buffers: shared hit=7
7	-> Nested Loop (cost=0.14..9.80 rows=1 width=430) (actual time=0.210..0.254 rows=3.00 loops=1)
8	Buffers: shared hit=7
9	-> Seq Scan on orders (cost=0.00..1.56 rows=1 width=90) (actual time=0.170..0.205 rows=3.00 loops=1)
10	Filter: (customer_id = 1)
11	Rows Removed by Filter: 42
12	Buffers: shared hit=1
13	-> Index Scan using restaurants_pkey on restaurants (cost=0.14..8.16 rows=1 width=340) (actual time=0.013..0.013 rows=1.00 loops=1)
14	Index Cond: (id = orders.restaurant_id)
15	Index Searches: 3
16	Buffers: shared hit=6
17	Planning Time: 0.376 ms
18	Execution Time: 0.328 ms

e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

SELECT

restaurants.name AS restaurant_name,

orders.total AS order_total,

orders.status AS order_status,

orders.created_at AS order_date

FROM orders

INNER JOIN restaurants ON orders.restaurant_id = restaurants.id

WHERE orders.customer_id = 1

ORDER BY orders.created_at DESC

LIMIT 5;

Data Output

Messages

Notifications

≡

+

📄

▼

🗑️

▼

🔒

🔒

📄

📄

📄

📄

📄

📄

SQL

	restaurant_name character varying (100) 🔒	order_total numeric (10,2) 🔒	order_status character varying (20) 🔒	order_date date 🔒
1	Radhe Dhosa	230.00	completed	2026-06-16
2	Balaji Pizza	420.00	completed	2026-06-08
3	Jay Khodiyar Locho	150.00	completed	2026-06-01

f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

Metrics: Before indexing, the database performed a full sequential scan inspecting all 45 rows followed by an explicit sort operation; after indexing, it shifted to a direct **Index Scan**, examining exactly 5 rows and reducing execution time by roughly **75%**.